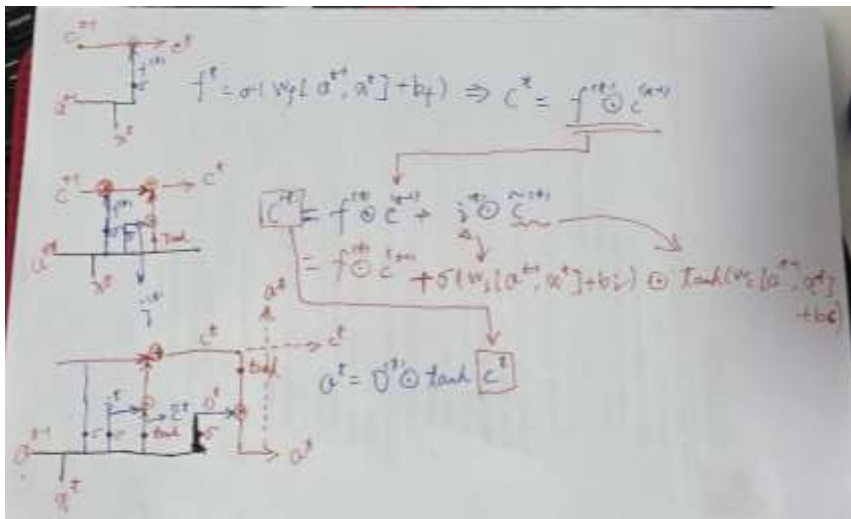


01. (视频 071 中) 请整理: 遗忘门、输入门、输出门、 $a^{<t>}$ 、 $c^{<t>}$ 是如何求得的。
要求: 把计算过程的公式写出来, 按照公式, 把图复现一遍。



答:

02. 在 CustomRNN 类的初始化方法中, W_{xa} 、 W_{aa} 、 W_{ah} 和偏置 b_a 、 b_h 被定义为 `nn.Parameter`。为什么要使用 `nn.Parameter`? 如果使用普通的 `torch.Tensor`, 会发生什么?

答: 优化器需要知道它应该更新哪些变量参数, 通过遍历 `model.parameters()` 来获取这些变量。因为使用了 `nn.Parameter`, 权重矩阵 W_{xa} 、 W_{aa} 、 W_{ah} 等就被包含在这个列表中了。如果使用普通的 `torch.Tensor`, 因为普通的 `Tensor` 不会被自动添加到 `model.parameters()` 中。当调用 `optimizer.step()` 时, 优化器根本不知道 W_{xa} 的存在, 因此不会更新它的值。

03. 为什么 `forward` 方法中的 `for t in range(x.size(1))` 会遍历所有的时间步? 可否改成 `for t in range(x.size(2))`? 如何如果可以的话, 应当如何理解?

答: x 的形状是 $(batch_size, seq_len, input_size)$ 。具体到 MNIST 数据集中:

- $x.size(0) = \text{Batch Size (64)}$
- $x.size(1) = 28$ (图像的高度, 被视为序列长度/时间步)
- $x.size(2) = 28$ (图像的宽度, 被视为每个时间步的特征维度)

`for t in range(x.size(1))` 选择的是按照图像“高度单元格”, 也就是“一行一行去看, 把图片的每一行数据当作一个时间步”, 所以这一句代码意思是:

- `range(x.size(1))` 意味着我们从图像的第 0 行遍历到第 27 行。
- 每次循环 `x_t = x[:, t, :]` 取出的是一行像素 (形状为 `[batch, 28]`)。
- `Size(1)` 这相当于 RNN 从上到下“扫描”这张图片(如下图所示)



如果是 `size(2)`，那就是“从左到右扫描（逐列处理），而不是从上到下”（如下图）



04. 在 `CustomRNN` 类中，`torch.mm(x_t, self.Wxa)`、`torch.mm(h_t, self.Waa)`、`self.ba` 的形状分别是怎样的？这样三个形状的对象，为什么可以进行 `torch.mm(x_t, self.Wxa) + torch.mm(h_t, self.Waa) + self.ba` 计算？如何完成的维度匹配？

答：

`torch.mm(x_t, self.Wxa)` 形状为 (64, 128)

`torch.mm(h_t, self.Waa)` 形状为 (64, 128)

`self.ba` 形状为 (128)

因为有 PyTorch 的广播机制 (Broadcasting)，所以可以相加，`ba` 偏置量缺少一个 `batch_size` 维度数据是一个一维向量，PyTorch 广播会自动将 `self.ba` 沿着 `Batch` 维度复制 64 次。

- 矩阵维度（前两项）：Batch (64) x Hidden (128)
- 偏置维度（+号最后一项）：(64, 广播得到) x Hidden (128)

05. 隐藏状态：`nn.RNN` 返回的隐藏状态和输出之间有什么区别？

`out, _ = self.rnn(x)` # (返回两个值，一个是 `output`，这一行里面用“`out`”代表，一个是 `h_n`，代码里面用“`_`”代表，然后在下一行代码里面实际只调用一个也就是 `out`)

`out = self.fc(out[:, -1, :])`

#这一行里面，`output` 包含了 `RNN` 最后一层在每一个时间步 (time step) 产生的隐藏状态，当设置了 `batch_first=True` 时，

`output` 此时形状为 (batch_size, seq_len, hidden_size)，因为是分类任务需要读取 28 行最后只有一个输出，只需要知道 `seq_len=27`，也就是最后一个时间步 `out[:, -1, :]` 数据来 `return` 出来“`out`”。

06. 模型参数: 在 nn.RNN 中, input_size、hidden_size 与我们自定义 RNN 中的 Wxa、Waa、Wah 在形状上有什么关系?

答: input_size(28): 对应 Wxa 的行数。在 MNIST 例子中, 每一行像素 (28 个像素点) 作为一个时间步的输入。

hidden_size(128): 它是 RNN 的“神经元”数量。它同时出现在 Wxa 的列、Waa 的行列以及 Wah 的行中。这保证了隐藏状态 s_{t-1} 可以不断自循环并与输入结合。

output_size(10): 对应 Wah 的列数, 决定了最终输出向量的长度 (对应 0-9 十个数字)。

07. 将使用 Jena_Climate 数据集进行的 LSTM 实战代码运行效果截图提交

